

Manual Completo: Roteador Inteligente de LLMs Gratuitas

Versão: 1.0.0 Data: 2026-08-17 Autor: Fábrica Agêntica de Publicações Arquivo: scripts/task_router.py

line(length: 100%)

Sumário

line(length: 100%)

1. O que é e por que existe

O problema

Você tem 13 provedores de LLM gratuitos configurados (Groq, Google, OpenRouter, Cerebras, etc.). Cada um tem: - Modelos diferentes (Llama, Gemini, Qwen, etc.) - Limites de cota diferentes (alguns estouram rápido, outros não) - Pontos fortes diferentes (Groq é rápido para código, Google é bom para raciocínio, etc.)

O problema: toda vez que você inicia uma sessão, precisa **decidir manualmente** qual provedor usar. Se escolher errado, perde tempo ou estoura a cota.

A solução

O task_router.py é um script que: 1. **Lê o que você quer fazer** (seu prompt) 2. **Detecta o tipo de tarefa** (código, raciocínio, criação, análise, chat, embedding, visão) 3. **Escolhe o melhor provedor** para aquela tarefa 4. **Define as variáveis de ambiente** (ORCA_PROVIDER e ORCA_MODEL) que o harness usa 5. **Faz fallback automático** se o provedor preferido estiver sem cota

Compatibilidade

Funciona com **qualquer harness** que leia variáveis de ambiente:

Harness	Funciona?	Como
Orca / MiMoCode	✓	ORCA_PROVIDER + ORCA_MODEL
Claude Code	✓	ANTHROPIC_API_KEY ou OPENAI_API_KEY via env
Aider	✓	OPENAI_API_KEY via env
Cursor	✓	Configura via UI, mas env serve como referência
Continue	✓	OPENAI_API_KEY via env
Windsurf	✓	Configura via UI, mas env serve como referência
Gemini CLI	✓	GEMINI_API_KEY via env
Qualquer CLI/terminal	✓	export direto

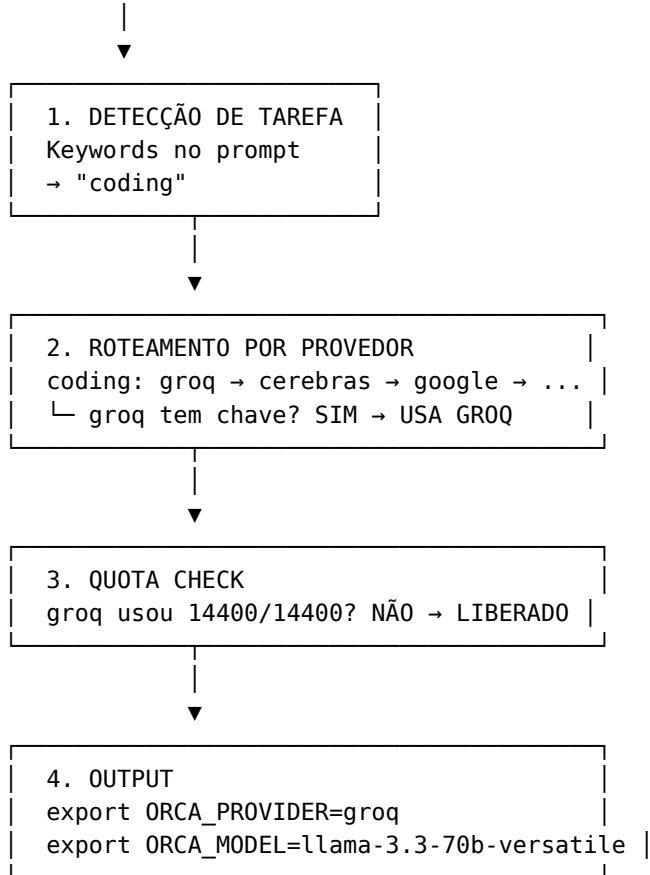
Não depende de nenhum harness específico. É um script Python puro que seta variáveis de ambiente.

line(length: 100%)

2. Funcionamento interno

Fluxo de decisão

Seu prompt: "debug this Python function"



Como detecta a tarefa

O script analisa palavras-chave no seu prompt. Exemplos:

Prompt	Tarefa detectada	Keywords que matcharam
"debug this Python function"	coding	debug, function, python
"escreva um poema criativo"	creative	escreva, poema, criativo
"analise estes dados CSV"	analysis	analise, dados, csv
"criar embedding de busca"	embedding	criar, embedding, busca
"descreva esta imagem"	vision	descreva, imagem
"bom dia, como vai"	chat	bom dia, como, vai
"por que o código falha"	reasoning	por que, código

Se nenhuma keyword matchar, o padrão é **chat**.

Por que esses provedores para cada tarefa

Tarefa	Provedor 首选	Razão
coding	Groq	Llama 3.3 70B é excelente em código, latência baixa
reasoning	Google	Gemini 1.5 Pro tem raciocínio avançado
creative	OpenRouter	Acesso a múltiplos modelos free para texto criativo
analysis	Google	Gemini Pro lida bem com dados e resumos
chat	Groq	Respostas rápidas para conversas leves
embedding	Hugging Face	Modelos de embedding gratuitos (BGE-M3)
vision	Google	Gemini Pro aceita imagens nativamente

line(length: 100%)

3. Pré-requisitos

O que você precisa ter

1. Python 3.8 ou superior

```
python --version
# Deve mostrar 3.8+
```

2. Arquivo .env com as chaves dos provedores

- Já existe no projeto: proj_fabrica-de-livros/.env
- Contém chaves de 13 provedores gratuitos

3. python-dotenv (recomendado)

```
pip install python-dotenv
```

Sem ele, o script lê direto do os.environ. Com ele, carrega o .env automaticamente.

O que NÃO precisa

- Não precisa de Docker
- Não precisa de servidor rodando
- Não precisa de nenhuma dependência pesada
- Não precisa de internet (só para chamar a API depois)

line(length: 100%)

4. Instalação passo a passo

Passo 4.1 — Instalar python-dotenv

Abra o terminal e rode:

```
pip install python-dotenv
```

Se der erro de permissão:

```
pip install --user python-dotenv
```

Passo 4.2 — Verificar que o .env existe

```
cat proj_fabrica-de-livros/.env | head -20
```

Deve mostrar as chaves dos provedores (GROQ_API_KEY, GEMINI_API_KEY, etc.).

Passo 4.3 — Testar o script

```
cd proj_fabrica-de-livros
python scripts/task_router.py --help
```

Deve mostrar a ajuda:

Uso: python scripts/task_router.py "seu prompt"

Opções:

```
--json      Saída em JSON
--info      Saída legível (PT-BR)
--shell     Saída em export shell (padrão)
--status    Mostra quotas e provedores ativos
--reset     Reseta quotas de todos os provedores
```

Passo 4.4 — Ver provedores ativos

```
python scripts/task_router.py --status
```

Deve mostrar todos os provedores com  ATIVO.

line(length: 100%)

5. Uso básico — seu primeiro roteamento

Exemplo 1: Roteamento simples

```
python scripts/task_router.py "debug this Python function"
```

Saída:

```
export ORCA_PROVIDER=groq
export ORCA_MODEL=llama-3.3-70b-versatile
```

O que aconteceu: o script detectou “coding” (keywords: debug, function, python), escolheu Groq como melhor provedor para código, e retornou os exports que definem o provider e modelo.

Exemplo 2: Ativar no shell

```
eval $(python scripts/task_router.py "debug this Python function")
```

O que aconteceu: o eval executa os exports, definindo ORCA_PROVIDER=groq e ORCA_MODEL=llama-3.3-70b-versatile no seu shell atual. Agora qualquer programa que leia essas variáveis vai usar Groq.

Exemplo 3: Ver em JSON

```
python scripts/task_router.py "escreva um poema" --json
```

Saída:

```
{
  "task": "creative",
```

```

    "provider": "openrouter",
    "model": "meta-llama/llama-3.3-70b-instruct:free",
    "fallback": false
}

```

Exemplo 4: Ver em PT-BR legível

```
python scripts/task_router.py "analise estes dados" --info
```

Saída:

```

Task: analysis
Provider: google
Model: gemini-1.5-pro

line(length: 100%)

```

6. Uso em cada harness

6.1 Orca / MiMoCode

O Orca lê ORCA_PROVIDER e ORCA_MODEL quando definidos.

Método 1 — Pré-sessão (recomendado):

```

# 1. Abra o terminal
cd D:
\Backup_C_trcnologia_2026-08-14\Desktop\01_Projetos_e_Desenvolvimento\proj_fabrica-
de-livros

# 2. Rote o router com sua tarefa
eval $(python scripts/task_router.py "refatore este código")

# 3. Inicie o Orca
orca

```

Método 2 — Alias permanente (PowerShell):

Adicione ao seu \$PROFILE (abra com notepad \$PROFILE):

```

function ai {
    param([string]$Prompt)
    $result = python D:
\Backup_C_trcnologia_2026-08-14\Desktop\01_Projetos_e_Desenvolvimento\proj_fabrica-
de-livros\scripts\task_router.py $Prompt
    $result | ForEach-Object {
        if ($_ -match 'export (\w+)=(.+)') {
            [Environment]::SetEnvironmentVariable($matches[1], $matches[2],
"Process")
        }
    }
    Write-Host "Provider: $env:ORCA_PROVIDER | Model: $env:ORCA_MODEL" -
ForegroundColor Green
}

```

Depois:

```

ai "debug this function"
# Provider: groq | Model: llama-3.3-70b-versatile
orca

```

Método 3 — Alias permanente (Git Bash / Linux / Mac):

Adicione ao ~/.bashrc ou ~/.zshrc:

```
alias ai='eval $(python D:/Backup_C_trcnologia_2026-08-14/
Desktop/01_Projetos_e_Desenvolvimento/proj_fabrica-de-livros/scripts/task_router.py)'
```

Depois:

```
source ~/.bashrc
ai "escreva um poema"
orca
```

6.2 Claude Code

O Claude Code usa ANTHROPIC_API_KEY. O router pode servir como referência:

```
# Descubra o melhor provedor
python scripts/task_router.py "minha tarefa" --info
# → Provider: groq | Model: llama-3.3-70b-versatile
```

```
# Use a informação para configurar no Claude Code
# Settings → Provider → Groq → cole a chave
```

Ou, se o Claude Code suportar OPENAI_API_KEY para provenviders compatíveis:

```
eval $(python scripts/task_router.py "minha tarefa")
export OPENAI_API_KEY=$GROQ_API_KEY # se usar Groq via OpenAI-compatible
claude
```

6.3 Aider

O Aider lê OPENAI_API_KEY:

```
eval $(python scripts/task_router.py "minha tarefa")

# Se o provedor escolhido for Groq, configure:
export OPENAI_API_BASE=https://api.groq.com/openai/v1
export OPENAI_API_KEY=$GROQ_API_KEY

aider
```

6.4 Cursor / Windsurf

Esses harnesses configuram provedores via UI. O router serve como referência:

```
python scripts/task_router.py "minha tarefa" --info
# → Provider: google | Model: gemini-1.5-pro
```

```
# Vá no Cursor: Settings → Models → Add Provider
# Selecione Google Gemini, cole a chave
```

6.5 Continue

O Continue lê OPENAI_API_KEY:

```
eval $(python scripts/task_router.py "minha tarefa")
export OPENAI_API_BASE=https://api.groq.com/openai/v1
export OPENAI_API_KEY=$GROQ_API_KEY
continue
```

6.6 Gemini CLI

```
eval $(python scripts/task_router.py "minha tarefa")
# O router já setou ORCA_PROVIDER=google
```

```
# Gemini CLI lê GEMINI_API_KEY diretamente
gemini
```

6.7 Terminal puro (sem harness)

```
eval $(python scripts/task_router.py "minha tarefa")
echo "Provider: $ORCA_PROVIDER"
echo "Model: $ORCA_MODEL"
# Agora use curl para chamar a API diretamente:
curl https://api.groq.com/openai/v1/chat/completions \
  -H "Authorization: Bearer $GROQ_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model": "llama-3.3-70b-versatile", "messages":
[{"role": "user", "content": "Olá"}]}'
```

line(length: 100%)

7. Referência completa de comandos

Roteamento

Comando	O que faz	Exemplo de saída
python scripts/task_router.py "prompt"	Roteia e imprime exports shell	export ORCA_PROVIDER=groq
python scripts/task_router.py "prompt" --json	Roteia e imprime JSON	{"task": "coding", "provider": "groq", ...}
python scripts/task_router.py "prompt" --info	Roteia e imprime PT-BR	Task: coding\nProvider: groq
python scripts/task_router.py "prompt" --shell	Roteia e imprime exports (padrão)	export ORCA_PROVIDER=groq

Gerenciamento

Comando	O que faz
python scripts/task_router.py --status	Mostra todos os provedores, quotas e uso
python scripts/task_router.py --reset	Reseta quotas de todos os provedores
python scripts/task_router.py --help	Mostra ajuda

Atalhos

Atalho	Equivalente
ai "prompt"	eval \$(python scripts/task_router.py "prompt") (se alias configurado)

line(length: 100%)

8. Tabela de roteamento por tarefa

Mapeamento completo

Tipo de Tarefa	Provedores (ordem de preferência)	Modelo Padrão	Quando usar
coding	groq → cerebras → google → openrouter → nvidia	llama-3.3-70b-versatile	Debug, refatoração, implementação, testes, scripts
reasoning	google → groq → nvidia → openrouter → cerebras	gemini-1.5-pro	Análise lógica, comparações, decisões, tradeoffs
creative	openrouter → google → groq → siliconflow → nvidia	llama-3.3-70b-instruct:free	Escrita criativa, copy, blog, newsletters, poesia
analysis	google → groq → cerebras → openrouter → nvidia	gemini-1.5-pro	Resumos, extração de dados, relatórios, métricas
chat	groq → google → openrouter → huggingface → siliconflow	llama-3.3-70b-versatile	Conversas leves, perguntas rápidas, ajuda geral
embedding	huggingface → siliconflow → cohere → nvidia → fireworks	Phi-3.5-mini-instruct	Vetores, busca semântica, RAG, clusterização
vision	google → openrouter → nvidia → siliconflow → fireworks	gemini-1.5-pro	Análise de imagens, screenshots, diagramas, UI

Limites diários dos provedores

Provedor	Limite Diário (estimativa)	Reset
Groq	14.400 requests	Diário
Cerebras	1.000.000 tokens	Diário
Google	1.500 requests	Diário
OpenRouter	50 requests (sem saldo)	Diário
NVIDIA	1.000 requests	Diário
SiliconFlow	1.000 requests	Diário
Hugging Face	1.000 requests	Diário
Cohere	1.000 requests	Mensal
Fireworks	1.000 requests	Diário
Cloudflare	10.000 neurônios	Diário
Mistral	1.000 requests	Diário
SambaNova	1.000 requests	Diário
GitHub	1.000 requests	Diário

line(length: 100%)

9. Gerenciamento de quotas

O que são quotas

Cada provedor tem um limite diário de uso gratuito. O router conta quantas vezes você usou cada provedor e, quando chega perto do limite, bloqueia temporariamente e usa o próximo da lista.

Onde fica o estado

```
~/.task_router/  
├─ quota.json      ← estado de quotas (JSON)  
└─ usage.log       ← log de uso (texto legível)
```

Ver quanto já usou

```
python scripts/task_router.py --status
```

Mostra uso: N para cada provedor. Se um provedor tiver 🟡 BLOQUEADO, significa que estourou a quota.

Resetar quotas

Se um provedor bloqueou mas a quota diária já liberou (geralmente à meia-noite UTC):

```
python scripts/task_router.py --reset
```

Isso zera todos os contadores.

Resetar um provedor específico

Não há comando direto, mas você pode editar manualmente:

```
# Windows  
notepad %USERPROFILE%\task_router\quota.json
```

```
# Linux/Mac  
nano ~/.task_router/quota.json
```

Delete a linha do provedor que quer resetar.

Log de uso

O `usage.log` registra cada uso:

```
2026-08-17T18:30:00+00:00 | groq | use #1
2026-08-17T18:31:00+00:00 | google | use #1
2026-08-17T18:32:00+00:00 | groq | use #2
```

Para ver o log:

```
cat ~/.task_router/usage.log
```

line(length: 100%)

10. Solução de problemas

Problema: “Erro: prompt vazio”

Causa: Não passou nenhum prompt.

Solução:

```
python scripts/task_router.py "sua tarefa aqui"
```

Problema: Saída mostra “fallback: true”

Causa: Nenhum provedor da lista preferida tem chave configurada.

Solução: 1. Verifique o `.env`: `cat .env | grep API_KEY` 2. Veja quais provedores estão ativos: `python scripts/task_router.py --status` 3. Adicione a chave do provedor desejado no `.env`

Problema: Provedor bloqueado (🟡 BLOQUEADO)

Causa: O provedor estourou a quota diária.

Solução:

```
python scripts/task_router.py --reset
```

Ou espere até a meia-noite UTC (quando as quotas resetam automaticamente).

Problema: python-dotenv não carrega o `.env`

Causa: python-dotenv não instalado.

Solução:

```
pip install python-dotenv
```

Ou exporte manualmente:

```
# Linux/Mac/Git Bash
export $(cat .env | grep -v '^#' | xargs)

# PowerShell
Get-Content .env | ForEach-Object {
    if ($_ -match '^(^#|=)(.*)$') {
        [Environment]::SetEnvironmentVariable($matches[1].Trim(),
        $matches[2].Trim().Trim('\"'), "Process")
    }
}
```

Problema: “command not found: python”

Causa: Python não está no PATH ou se chama python3.

Solução:

```
# Tente python3
python3 scripts/task_router.py --status
```

```
# Ou verifique onde está
which python    # Linux/Mac
where python    # Windows
```

Problema: Orca não muda de modelo

Causa: O Orca locka o modelo na inicialização.

Solução: Rote antes de iniciar o Orca:

```
eval $(python scripts/task_router.py "minha tarefa")
orca # inicia com o modelo novo
```

Não adianta rodar o router depois que o Orca já começou.

Problema: Modelo não é suportado pelo harness

Causa: O router define um modelo que o harness não conhece.

Solução: Use --info para ver o modelo e selecione manualmente no harness:

```
python scripts/task_router.py "minha tarefa" --info
# → Provider: groq | Model: llama-3.3-70b-versatile
# Vá no harness e selecione groq/llama-3.3-70b-versatile manualmente
```

```
line(length: 100%)
```

11. Perguntas frequentes

Posso usar com mais de um harness ao mesmo tempo?

Sim. O router seta variáveis de ambiente no shell. Qualquer programa que leia essas variáveis vai usá-las. Se você tem dois terminals abertos, cada um pode ter um provider diferente.

O router consome tokens/chamadas?

Não. O script roda localmente e não faz nenhuma chamada de API. Ele só lê variáveis de ambiente e imprime na tela.

Posso customizar os provedores por tarefa?

Sim. Edite o dicionário TASK_ROUTES no script:

```
TASK_ROUTES = {
    "coding": ["groq", "cerebras", "google"], # mude a ordem ou adicione provedores
    ...
}
```

Posso adicionar um provedor novo?

Sim. Adicione nas 4 seções do script: 1. TASK_ROUTES — ordem de preferência por tarefa 2. PROVIDER_MODELS — modelos gratuitos 3. ENV_VARS — variável de ambiente da chave 4. No .env — a chave do provedor

O router funciona sem internet?

Sim. O script roda 100% local. A internet só é necessária depois, quando você chama a API do provedor.

Como vejo o histórico de uso?

```
cat ~/.task_router/usage.log
```

Mostra data/hora, provedor e número de uso de cada chamada.

Posso desabilitar a quota tracking?

Sim. Delete ou comente as linhas de `_registrar_uso` e `_provider_bloqueado` no script. O roteamento continua funcionando, só não conta usage.

Qual a diferença entre `-shell` e `-info`?

- `--shell` (padrão): imprime `export ORCA_PROVIDER=...` — pronto para eval
- `--info`: imprime texto legível em PT-BR — bom para ler no terminal
- `--json`: imprime JSON estruturado — bom para scripts e automação

`line(length: 100%)`

Arquivos Relacionados

Arquivo	Caminho	Descrição
Script principal	<code>scripts/task_router.py</code>	O roteador em si
Detector de provedores	<code>scripts/detectar_llms_gratuitas.py</code>	Mapeia provedores ativos (anterior)
Chaves dos provedores	<code>.env</code>	13 chaves de provedores gratuitos
Quotas	<code>~/.task_router/quota.json</code>	Estado de quotas por provedor
Log de uso	<code>~/.task_router/usage.log</code>	Histórico de chamadas
Plano de implementação	<code>melhorias/2026-08-17-roteador-inteligente-llm.md</code>	Documento de design
Este manual	<code>docs/manual-roteador-inteligente-llm.md</code>	Este arquivo